

DOKUMENTACJA TECHNICZNA

Harmonogram spłat

Architektura, API, domena, interfejs i testy

Dokument dla zespołu IT utrzymującego i rozwijającego aplikację.

Wersja: MVP

Data: 23 września 2026

Stack: Next.js App Router, TypeScript, React, Tailwind, Vitest

1. Zakres i architektura

Aplikacja składa się z czterech warstw:

Warstwa	Plik	Odpowiedzialność
Dane	<code>src/dane/wskazniki.ts</code> oraz <code>dane/*.json</code>	Import i udostępnienie serii POLSTR 1M i WIBOR 3M.
Domena	<code>src/domena/harmonogram.ts</code>	Czyste obliczenia finansowe, typy, walidacja domenowa i daty.
API	<code>app/api/harmonogram/route.ts</code>	Parsowanie query string, konwersja jednostek, wywołanie domeny i JSON.
UI	<code>app/page.tsx</code> , <code>app/globals.css</code>	Formularz, fetch, stany ekranu, tabela, formatowanie i eksport CSV.

Logika domenowa nie importuje Reacta, nie czyta plików i nie wykonuje I/O. Route handler nie zawiera pętli obliczeniowej.

2. Technologie i uruchomienie

- Node.js 22 lub nowszy zgodnie z `package.json`.
- Next.js 16 z App Routerem.
- React 19 i TypeScript strict.
- Tailwind CSS przez PostCSS.
- Vitest do testów.

```
npm install
npm run dev
npm test
npm run typecheck
npm run lint
npm run build
```

3. Moduł domenowy

Typy publiczne

Typ	Znaczenie
<code>ParametryKredytu</code>	Kwota w groszach, liczba rat, marża jako ułamek, typ rat, wskaźnik, data, seria i nadpłaty.
<code>WpisSerii</code>	<code>od: string</code> oraz <code>stopa: number</code> jako ułamek, np. 0,0355.
<code>Nadplata</code>	<code>numerRaty</code> , <code>kwotaGr</code> i tryb <code>obniz_rate</code> lub <code>skroc_okres</code> .
<code>Rata</code>	Numer, data, oprocentowanie oraz wszystkie kwoty w groszach.
<code>WynikHarmonogramu</code>	Lista rat i <code>sumaOdsetekGr</code> .

Funkcje

Funkcja	Działanie
<code>policzHarmonogram</code>	Główna funkcja domenowa. Waliduje parametry, generuje raty, stosuje serię wskaźnika i nadpłaty oraz zwraca wynik.
<code>zaokrąglijGrosze</code>	Jedno miejsce zaokrąglania wartości finansowych do całych groszy.
<code>dataJakoObiekt</code>	Waliduje datę RRRR-MM-DD i tworzy obiekt daty w UTC.
<code>dodajMiesiace</code>	Wyznacza datę kolejnej raty, zachowując poprawność dla krótszych miesięcy.
<code>dataDoTekstu</code>	Serializuje datę do RRRR-MM-DD.
<code>znajdzWskaźnik</code>	Wybiera ostatni wpis serii, którego data <code>od</code> nie przekracza daty raty.
<code>sprawdzParametry</code>	Sprawdza dodatnie kwoty i liczby, marżę, daty, serię oraz granice nadpłat.
<code>rataRowna</code>	Oblicza annuitet: saldo razy stopa okresowa przez mianownik wzoru renty.

4. Algorytm obliczeń

1. Domena rozpoczyna od salda równego kwocie kredytu oraz liczby pozostałych rat.
2. Dla daty każdej raty wybierany jest wskaźnik z serii i dodawana jest marża.
3. Odsetki to `zaokrąglij(saldo * oprocentowanie / 12)`.
4. Rata równa jest wyliczana na początku harmonogramu i ponownie po zmianie oprocentowania lub nadpłacie w trybie obniżenia raty.
5. Rata malejąca używa stałej części kapitałowej, a odsetki maleją wraz z saldem.
6. W ostatniej racie kapitał jest ustawiany na całe pozostałe saldo.
7. Nadpłata jest stosowana po racie i zmniejsza saldo. Tryb `obniz_rate` wymusza ponowne wyliczenie raty, a `skroc_okres` zachowuje dotychczasową ratę.
8. Pętla kończy się po wyzerowaniu salda lub po pierwotnej liczbie rat.

5. Źródło danych wskaźników

`src/dane/wskaźniki.ts` importuje JSON i udostępnia funkcję `seriaWskaźnika(wskaźnik)`. Wartości są przechowywane jako ułamki dziesiętne. Dane są przykładowe i nie mogą być traktowane jako komercyjne źródło referencyjne.

- POLSTR 1M: seria miesięczna.
- WIBOR 3M: seria kwartalna.
- Po ostatnim wpisie używana jest ostatnia znana wartość.

6. API

GET /api/harmonogram

Parametry podstawowe:

```
/api/harmonogram?kwota=40000&liczbaRat=300&marza=2.11&wskaznik=POLSTR_1M&typRat=rowne&pierwszaRata=2026-10-01
```

Nadpłaty przekazuje się wielokrotnym parametrem w formacie `numerRaty:kwota:tryb`, np. `nadplata=24:20000:skroc_okres`. Kwoty query są w złotych, a route konwertuje je na grosze.

Odpowiedź 200

```
{
  "raty": [{
    "numer": 1,
    "data": "2026-10-01",
    "kapital": 60805,
    "odsetki": 188667,
    "rata": 249472,
    "saldoPoSplacie": 39939195
  }],
  "sumaOdsetek": 34841480
}
```

Kwoty w JSON są groszami. Błędne parametry dają status 400 z polem `bład`.

7. Interfejs użytkownika

- Formularz utrzymuje stan parametrów kredytu oraz listę nadpłat.
- `oblicz` buduje `URLSearchParams`, wykonuje fetch i obsługuje wynik lub błąd.
- `formatujZl` konwertuje grosze na PLN przez `Intl.NumberFormat`.
- `eksportujCsv` tworzy plik UTF-8 z BOM, separatorem `;` i polskimi nagłówkami.
- Ekran obsługuje stany: pusty, ładowanie, wynik i błąd.
- Tabela pokazuje pierwsze 24 raty, z możliwością rozwinięcia całego harmonogramu.

8. Testy

Testy znajdują się w `tests/smoke.test.ts` i obejmują dane oraz domenę:

- ratą równą przy stałej stopie z kontrolą 2 494,72 zł i 2 492,53 zł na końcu,
- raty malejące i wyzerowanie salda,
- zmianę oprocentowania w trakcie harmonogramu,
- nadpłatę obniżającą ratę,
- nadpłatę skracającą okres,
- sumę kapitału i nadpłat równą kwocie kredytu,
- strukturę i uporządkowanie serii danych.

Weryfikacja projektu: `npm test`, `npm run typecheck`, `npm run lint`, `npm run build`.

9. Utrzymanie i rozszerzenia

- Każda zmiana algorytmu wymaga testu z konkretną liczbą kontrolną.
- Nie przenosić obliczeń do route handlera ani komponentu React.
- Potencjalne rozszerzenia: dzienne składanie POLSTR, prowizja za nadpłatę, RRSO, okresowo stała stopa i testy E2E.
- Przed wdrożeniem produkcyjnym należy zaktualizować dane referencyjne i zweryfikować ich źródło.